

- 第三步：虚拟机监控器将此页所属虚拟机的第二阶段页表相关页表项设置为 INVALID。
- 第四步：虚拟机监控器将此页设置为“空闲”或直接分配给其他虚拟机。

如果虚拟机再次访问该页面，将触发一次第二阶段缺页异常，并唤醒虚拟机监控器。此时虚拟机监控器查询第二阶段页表，发现此客户物理地址对应的页表项为 INVALID。然后查询该页对应的换页信息，最终将数据从存储设备拷贝回内存，更新第二阶段页表并恢复虚拟机的执行。

然而，与操作系统的换页机制相比，虚拟机监控器的换页会遇到语义鸿沟（Semantic Gap）的问题。为什么会有语义鸿沟？虚拟机监控器只能“看到”寄存器或内存中的二进制存储数据，它无法将这些二进制转化成有意义的虚拟机信息。具体来说，在换页的场景下，虚拟机监控器很难知道虚拟机内的哪些页在未来一段时间内不会被使用，也很难知道哪些页在下一刻会被使用。因此，虚拟机监控器难以高效地提供透明的换页机制，从而将真正不用的内存保存到存储设备中。最极端的情况是，虚拟机监控器刚将某虚拟机的内存页换页到存储设备，不久，客户操作系统也决定将该页换页到存储设备。此时虚拟机监控器首先需要将该页的数据重新从存储设备加载回内存，并恢复第二阶段页表的映射，之后虚拟机读取该内存页数据，最后把数据保存至存储设备中。

导致这种低效率的原因是虚拟机内部缺乏和虚拟机监控器的沟通机制。一种解决方案是让客户操作系统通知虚拟机监控器哪些内存存在未来一段时间内不会被使用。具体来说，可使用半虚拟化的方式为客户操作系统提供超级调用的接口，并修改客户操作系统内核中的代码，让它不断（或在某一个特定时刻）通知虚拟机监控器哪些内存一段时间内不会被使用。通过这种方式，虚拟机监控器就可以有针对性地将这些内存页换页到存储设备中。尽管这种方式有效，但是会对操作系统内核做较大的修改，因此不是一种优雅的方法。

真正优雅的解决方案是内存气球（Memory Ballooning）。这种方案的关键思路是在客户操作系统内核插入一个伪装的驱动。该驱动不会为内核其他模块提供任何有意义的功能，它的实际目的是和虚拟机监控器互相配合，并根据虚拟机监控器的要求，不断使用客户操作系统内核的接口分配或释放内存。之后，气球驱动将分配内存的客户物理地址通知虚拟机监控器，由于这些内存页都是气球驱动独占的空闲页面，因此虚拟机监控器无须保存或恢复这些内存，可以直接将其交给其他虚拟机或其他系统服务使用。

内存气球的具体使用方式如图 14.17 所示。首先，虚拟机监控器需要给虚拟机 2 分配更多内存，因此它通知虚拟机 1 中的气球驱动，让该驱动调用客户操作系统提供的内存分配接口分配大量内存（这一步很像气球膨胀的过程）。这些内存并不会被气球驱动直接使用，在得到这些内存之后，驱动会将这些内存对应的客户物理地址发送给虚拟机监控器。此后虚拟机监控器将虚拟机 1 传递的内存映射给虚拟机 2，并通知虚拟机 2 中的气球驱动。由于此气球驱动曾经执行气球膨胀的步骤，所以它会将之前分配的内存通过接口释放给操作系统内核（这一步很像气球收缩的过程）。



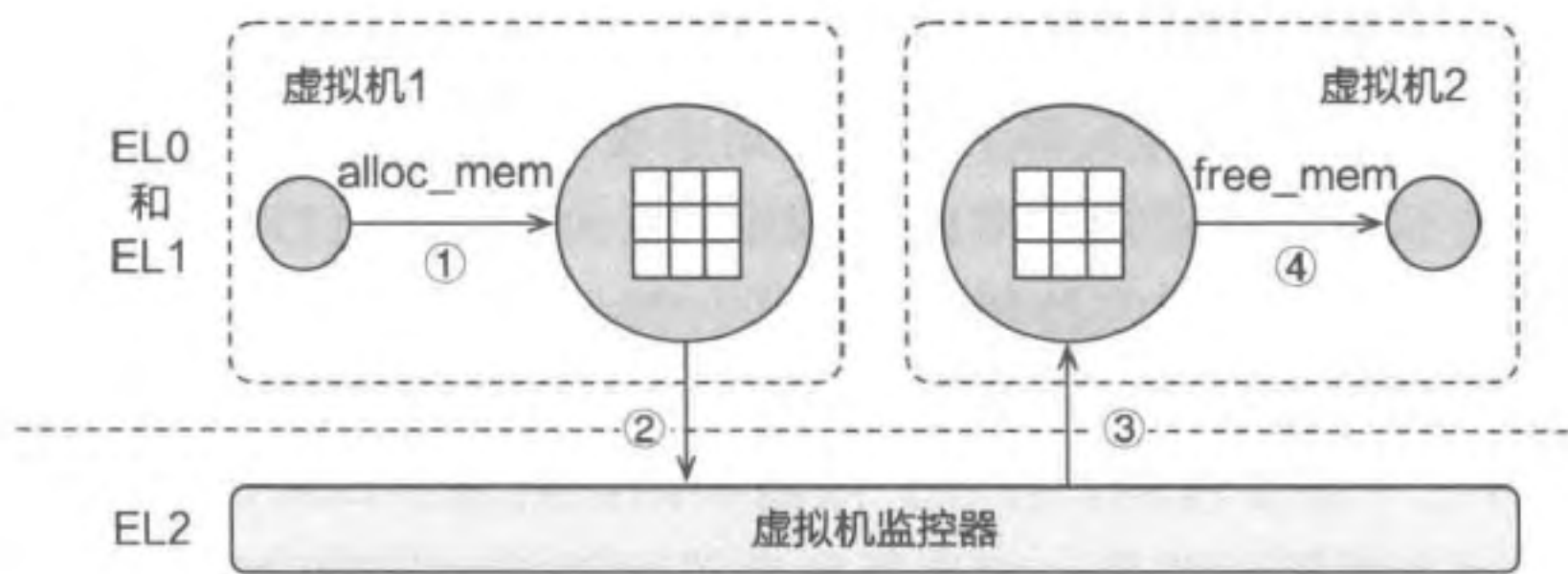


图 14.17 内存气球机制

内存气球方式之所以有效，是因为它巧妙地直接利用操作系统内核提供的内存管理接口。分配的内存都是操作系统内核维护的、未被任何模块使用的内存。这些内存存在被气球驱动得到之后，在未来一段时间内也不会被其他模块使用，因此可以安全地交给虚拟机监控器。通过这种方式，可以有效地避免之前遇到的两次换页的情况。此外，气球驱动器的写法非常简单，仅仅需要调用操作系统提供的接口，无须对操作系统内核做较大修改。因此，内存气球的方法既简单又高效。

14.4.5 小结

在本节，我们主要介绍了实现内存虚拟化的三种技术，这些技术依然可以分为两大类，如表 14.4 所示。在硬件虚拟化技术出现之前，主要通过影子页表和直接页表映射两种机制实现内存虚拟化，这两种机制在使用时也要与 14.3 节介绍的技术相配合。例如，影子页表机制需结合软件模拟的方法以便捕捉“安装页表”或“修改页表”的指令，直接页表映射机制则需与半虚拟化技术提供的超级调用接口同时使用。在硬件虚拟化技术出现之后，内存虚拟化主要依托于硬件提供的第二阶段地址翻译机制，第一阶段页表与第二阶段页表的维护被分开。

表 14.4 内存虚拟化技术总结

| 软件技术   | 硬件虚拟化技术  |
|--------|----------|
| 影子页表   | 第二阶段地址翻译 |
| 直接页表映射 |          |

14.5 I/O 虚拟化

本节主要知识点

- ❑ I/O 虚拟化的功能是什么？
- ❑ 如何通过软件模拟、半虚拟化、设备直通的方式实现 I/O 虚拟化？
- ❑ 上述三种技术的优缺点分别是什么？
- ❑ SR-IOV 和 IOMMU 是如何工作的？

操作系统的重要职责之一是管理设备。即使运行在虚拟机内，客户操作系统也依然“认为”自己需要并有能力管理设备。然而，出于安全和资源利用率的考虑，虚拟机监控器通常并不允许虚拟机直接管理真实的物理设备，而是使用 I/O 虚拟化技术为虚拟机提供一个“假”设备，例如虚拟磁盘和虚拟网卡，然后虚拟机可以像操作真的设备那样去操作这些虚拟设备。

I/O 虚拟化有三个重要功能。首先，限制虚拟机对真实物理设备的直接访问。物理设备通常需要被多个虚拟机共享，如果某台虚拟机能接触物理设备中的数据，将严重威胁其他虚拟机甚至宿主机的安全。试想一下，如果某台虚拟机有权限直接管理物理硬盘，那么直接格式化硬盘会如何（假设系统只有一个硬盘）？其次，为每个虚拟机提供虚拟设备接口。以磁盘设备举例，磁盘的起始扇区位置通常存储着重要信息，例如主引导记录（Master Boot Record, MBR）或全局唯一标识分区表（GUID Partition Table, GPT）。I/O 虚拟化为虚拟机提供虚拟设备的接口，例如将虚拟机对虚拟 1 号扇区的访问请求翻译至物理磁盘设备的对应扇区，从而读取 MBR 数据。最后，提高物理设备的资源利用率。以网卡设备为例，一台虚拟机也许不能占满物理网卡的网络带宽，但如果有多台虚拟机同时使用网卡，就可以有效地提高网卡的带宽利用率。

I/O 虚拟化主要有三种实现方式：软件模拟（全虚拟化）方法、半虚拟化方法以及设备直通方法。这三种方法各有优缺点，下面我们将依次介绍。

### 14.5.1 软件模拟方法

软件模拟的 I/O 虚拟化需借助模拟或硬件虚拟化方法捕捉原生驱动（即未经修改的与设备配备的驱动程序）的硬件指令，之后在虚拟机监控器内模拟虚拟设备的行为，并为虚拟机提供 I/O 服务。软件模拟的 I/O 虚拟化通常作用于机器指令层，它不要求对驱动程序做任何修改，客户操作系统可使用原生驱动程序与设备进行交互。因此软件模拟的 I/O 虚拟化方法也属于全虚拟化技术范畴。

对于磁盘设备，一种常见的软件模拟方案是虚拟机监控器将文件作为一台虚拟机的虚拟磁盘，并将虚拟磁盘的扇区位置映射为文件内的偏移量。因此，读取或写入虚拟磁盘的某个扇区会被翻译成对文件相关偏移位置的数据访问。

我们已经学习了操作系统与设备交互的三种方法：内存映射 I/O（Memory Mapped I/O, MMIO）、直接内存访问（Direct Memory Access, DMA）、中断（Interrupt）。基于软件模拟的 I/O 虚拟化方法需要捕捉客户操作系统对虚拟设备的每次请求，并采用软件的方法模拟上述三种交互。我们以主流虚拟机监控器 QEMU/KVM<sup>①</sup>中的网卡虚拟化为例来介绍这项技术。KVM 捕捉每次客户操作系统对虚拟网卡的请求，并将其中大部分请求转发到 QEMU 中。QEMU 之后为虚拟机提供虚拟网卡的模拟。例如，如果客户操作系统需要

---

① 我们将在 14.7 节对 QEMU 和 KVM 展开更详细的介绍。



发送网络包，QEMU 应该通过模拟 DMA 的方式从虚拟机内存中读取网络包数据，并借助物理网卡发送出去。由于不同网卡中的 MMIO 寄存器布局以及功能不同，因此 QEMU 需要针对不同网卡（如 RTL8139、Intel E1000）提供相应的模拟。

图 14.18 展示了 QEMU/KVM 通过软件模拟实现网络 I/O 虚拟化的简要步骤：

- 第一步：客户操作系统的网络协议栈在内存中生成网络包数据，并调用网卡原生驱动。
- 第二步：网卡驱动写入网卡 MMIO 地址，触发发送网络包的操作并生成一次 DMA 写请求。由于 MMIO 的客户物理地址已经在第二阶段页表中被映射成缺页，所以每次 MMIO 都会触发缺页异常并下陷进入 KVM 中。
- 第三步：KVM 读取引发下陷的客户物理地址，发现虚拟机正在执行 DMA 写操作，于是将这次引发下陷的客户物理地址等信息写入与 QEMU 进程共享的内存区域（KVM 中的 `run->mmio.data` 指向的内存），之后将控制流转至用户态的 QEMU。
- 第四步：QEMU 根据引发 MMIO 的客户物理地址发现访问的 MMIO 地址属于某种虚拟网卡。因为 QEMU 进程和虚拟机共享内存，所以它可以读取此 MMIO 内存中存储的 DMA 命令与地址。注意此处的 DMA 并不是物理 DMA，而是通过 QEMU 模拟的 DMA 操作——QEMU 直接从虚拟机内存中读取第一步网卡驱动生成的网络包数据。
- 第五步：用户态 QEMU 发起系统调用，将数据传入宿主机内核。
- 第六步：内核将数据发送至物理网卡的驱动程序，并借助物理网卡发送出去。
- 第七步：控制流回到用户态 QEMU。
- 第八步：QEMU 发现数据已经发送出去，决定向虚拟机中插入虚拟中断<sup>Ⓐ</sup>，以通知其 DMA 操作已经完成。最终 KVM 恢复虚拟机的执行，同时插入虚拟中断。

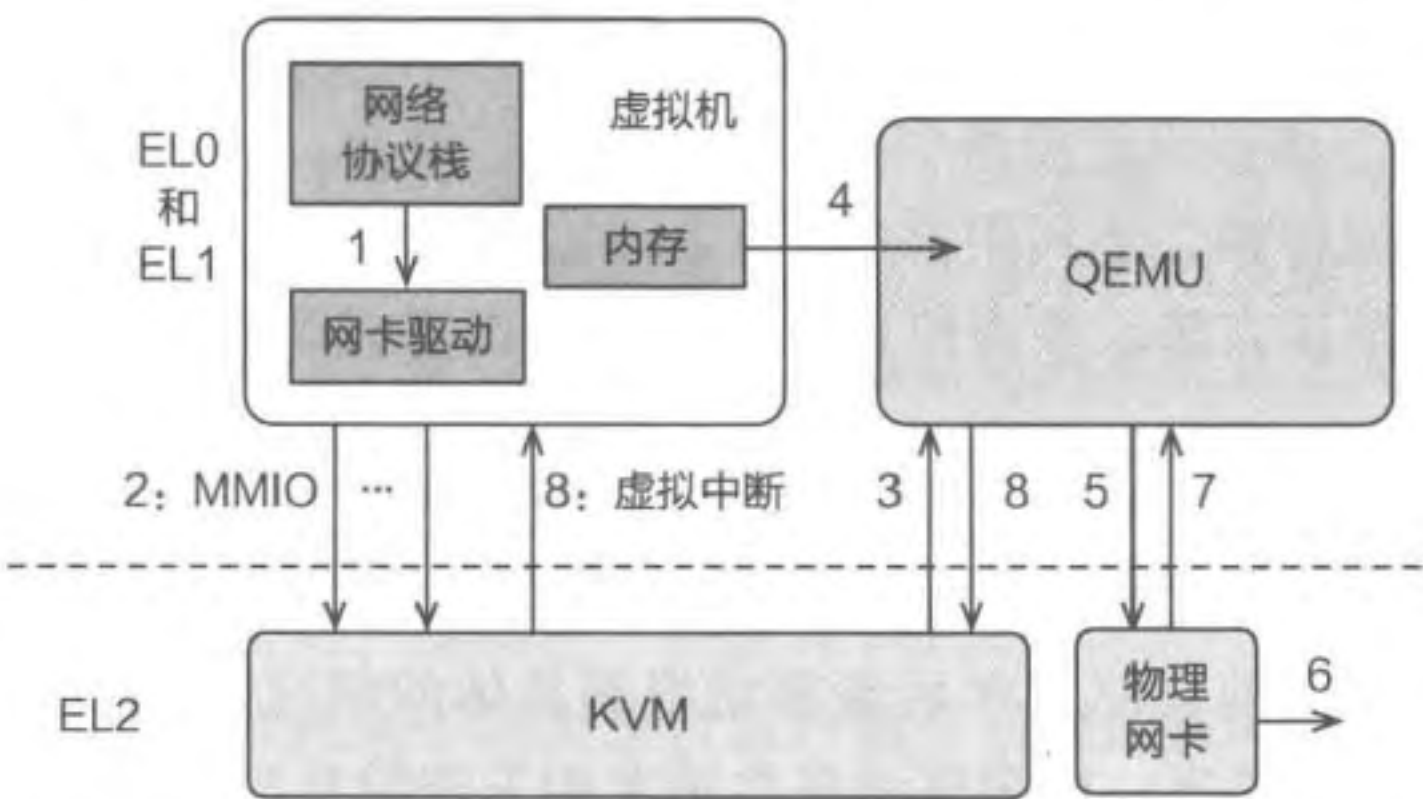


图 14.18 QEMU/KVM 通过软件模拟实现网络 I/O 虚拟化

Ⓐ 插入虚拟中断指虚拟机监控器通过软件模拟或硬件虚拟化调用客户操作系统注册在 `VBAR_EL` 中的中断处理函数。

### 小知识：Linux 虚拟网络设备——TAP 虚拟网卡与 Bridge 交换机

内核接收到 QEMU 的数据后，还需将网络包发送到以太网中，此过程通常借助 Linux TAP 虚拟网卡和 Bridge 交换机来实现。TAP 设备是在 Linux 内核中用软件实现的“假”网卡，虽然是由软件实现的，但也具备“真”网卡的功能。例如，它既能发送网络包也能接受网络包，还可以配置 IP 地址。TAP 设备与物理网卡不同的地方在于网络包的产生位置。对于物理网卡而言，网络包由本机之外的其他机器产生，物理网卡接收到之后交给驱动程序和网络协议栈处理。而对于 TAP 设备而言，网络包由某个用户态进程产生，发送给 TAP 之后再由驱动程序和网络协议栈处理。因此，TAP 设备包含两个部分，一个是字符设备，另一个是网络设备。字符设备负责与用户态交互，由用户态提供 TAP 设备的网络包；而网络设备则负责接入 Linux 的网络设备管理模块，比如可以接入 Linux Bridge 交换机。当 Linux Bridge 同时与物理网卡以及 TAP 网卡连接时，来自 TAP 的网络包可通过 Bridge 转发给物理网卡，并通过物理网卡发送至以太网。

由于软件模拟的 I/O 虚拟化方法直接作用于机器指令层，因此不需要修改操作系统和驱动程序的源代码，可直接使用已有的驱动程序。然而，如图 14.18 所示，每次 MMIO 的读写都会造成虚拟机下陷，所以该方法在使用虚拟设备运行时会带来较大的性能开销。

### 14.5.2 半虚拟化方法

为了优化 I/O 虚拟化的性能，我们可使用 14.3.5 节介绍的半虚拟化方法。在半虚拟化方法中，虚拟机不需要使用与特定设备对应的原生驱动程序，而是安装更为高效的前端驱动程序。前端驱动不使用会引起大量虚拟机下陷的 MMIO 接口，而是使用 PV (Para-virtualization) 接口与虚拟机监控器中的后端驱动进行交互。前后端驱动之间不仅通过共享内存机制进行数据传输，还利用批处理 (Batching) 方法将多次 I/O 请求整合为一次。因此，相对于软件模拟方法，半虚拟化方法减少了虚拟机下陷的次数，提升了 I/O 虚拟化的性能。

我们以 QEMU/KVM 架构下的 Virtio 为例解释半虚拟化方法的具体实现。在 Virtio 中，前端驱动和后端驱动之间的共享内存机制为 Virtqueue，其以队列的形式组织前后端驱动之间的共享内存。前端驱动和后端驱动根据具体的情况分别从队列两端读写信息。例如对于一次磁盘写入请求，前端驱动将此请求相关的信息写入 Virtqueue，而后端驱动从 Virtqueue 中读取信息并得到最终写入的数据。

图 14.19 展示了 KVM 中虚拟网络包的发送流程：

- 第一步：前端驱动程序将多次网络包数据按序写入 Virtqueue。



- 第二步：前端驱动程序发起超级调用<sup>⊖</sup>，造成虚拟机下陷，控制流进入 KVM。
- 第三步：KVM 读取超级调用参数，并将控制流转发至用户态的 QEMU 进程。
- 第四步：QEMU 根据 KVM 传入的信息，直接从共享的 Virtqueue 中读取描述符信息，得到这些网络包所在的客户物理地址。QEMU 将这些地址转化成对应的主机虚拟地址，最终得到网络包的数据。
- 第五步：用户态 QEMU 发起系统调用，将数据传入宿主机内核。
- 第六步：内核将数据发送至物理网卡驱动，并借助物理网卡发送出去。
- 第七步：控制流回到用户态 QEMU。
- 第八、九步：控制流回到虚拟机中，并插入虚拟中断。

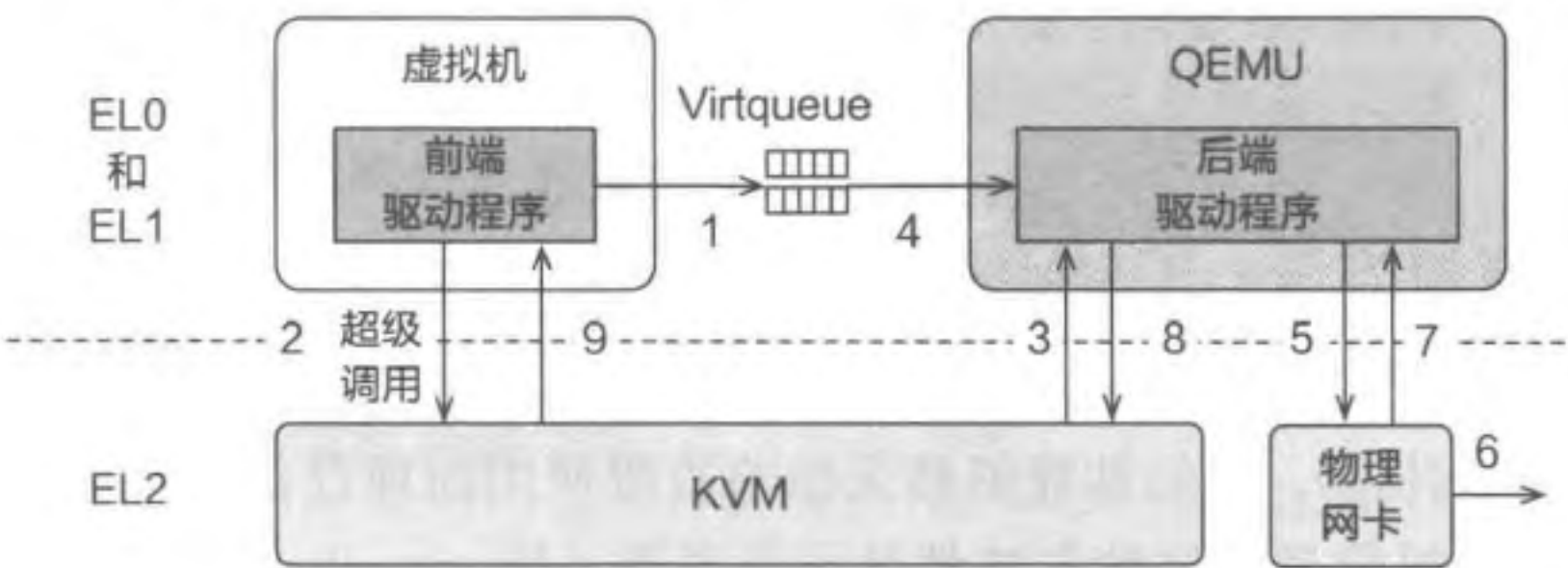


图 14.19 KVM Virtio 利用半虚拟化方法实现 I/O 虚拟化

读者可能会感到奇怪，半虚拟化流程和全虚拟化流程差不多，数据传输也是通过共享内存完成，为什么它的性能会优于全虚拟化方法？原因可分为两个方面。一方面，软件模拟（全虚拟化）方法会带来较大的性能开销，是因为它需要使用设备提供的 MMIO 接口。不仅客户操作系统每次读写 MMIO 会造成虚拟机下陷，而且多次 MMIO 访问只能传输少量的数据（例如一个网络包）。而在半虚拟化中，多次网络包数据可同时被放置于共享内存中，而这些数据传输只需使用一次超级调用。这种批量处理的方式减少了虚拟机下陷的次数，从而提高了 I/O 虚拟化的性能。另一方面，在半虚拟化方法中，同一类设备通常共享一组前后端驱动，不再需要为具体设备维护对应驱动，因而大大减少了驱动的数目。此外，简单的 PV 接口也使得前后端驱动的实现复杂度降低，因而更不易包含安全漏洞。因为驱动数目较少且驱动简单，另一种半虚拟化 I/O 实现方法是将后端驱动直接运行在 EL2 中，如图 14.20 所示的 Linux vhost 架构。这种架构减少了 KVM 与 QEMU 之间的特权级切换次数，能够进一步提升 I/O 虚拟化的性能。

半虚拟化方法的缺点依然在于需要修改客户操作系统的源码。为了添加前端驱动程序，半虚拟化方法需要修改客户操作系统，这对于源码不开源的操作系统来说较为困难。

⊖ 实际上这里的超级调用可以用其他方式替代，例如访问前后端驱动约定好的 MMIO 地址，由于该地址未在第二阶段页表中映射，会触发一次虚拟机下陷。

不过目前 Virtio 已经成为一种标准，越来越多的操作系统（例如 Windows）已经提供对 Virtio 的支持，因而半虚拟化方法的 I/O 虚拟化得到了越来越广泛的认可。

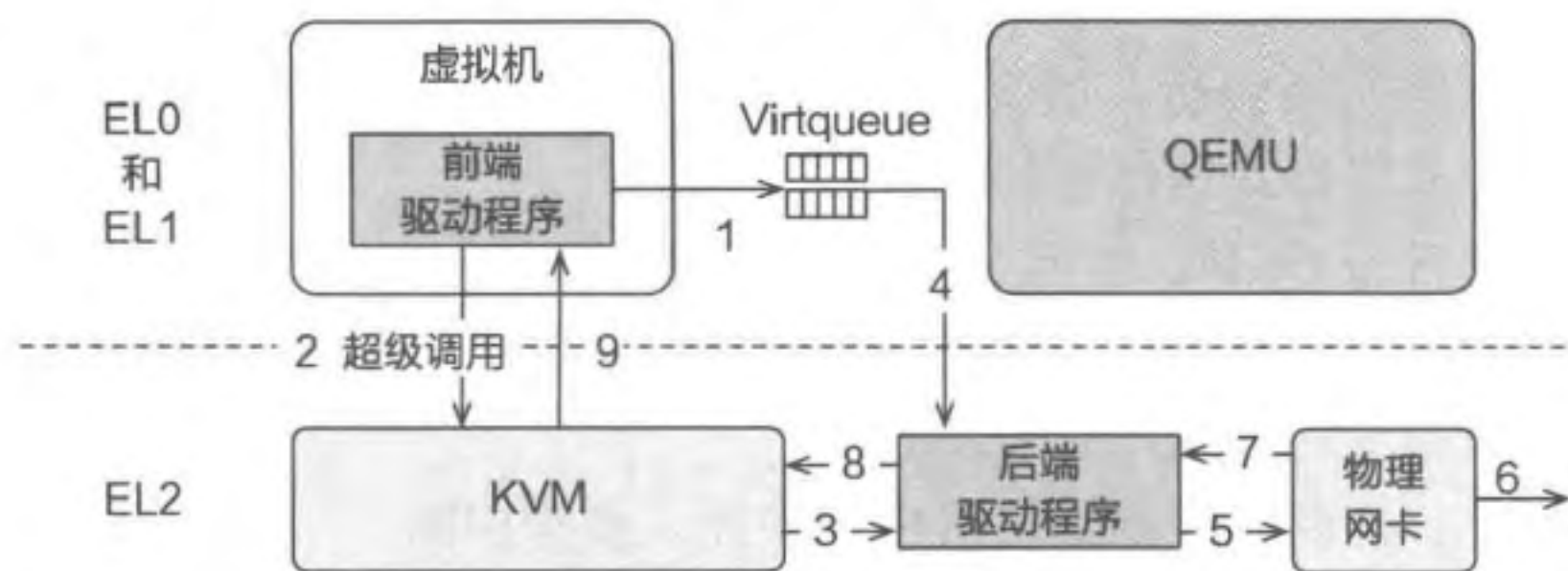


图 14.20 vhost-net 架构下的 I/O 虚拟化。后端驱动直接运行在 Linux 内核中，减少 I/O 路径上的特权级切换次数

14.5.3 设备直通方法：IOMMU 和 SR-IOV

尽管半虚拟化方法已经具备较好的性能，但因为在数据传输过程中偶尔会发生虚拟机下陷（超级调用引起），它的性能依然无法与直接使用物理设备相比。我们能否直接将物理设备交给虚拟机管理？这种方法就是设备直通（Device Passthrough，也称为设备透传）的 I/O 虚拟化方法。设备直通意味着物理设备从虚拟机监控器的管理中脱离，直接由虚拟机中的原生驱动程序进行管理。这种虚拟化方法的优势显而易见：虚拟机在使用设备时不需要任何虚拟机监控器的参与，所以与软件模拟或半虚拟化方法相比，该方法大大提升了 I/O 虚拟化的性能。

如图 14.21 所示，物理主机上有一块多余的存储设备。在系统启动时，该设备由虚拟机监控器的驱动程序管理。为了加速 I/O 虚拟化，虚拟机监控器选择将此存储设备直通给虚拟机 A 管理。而虚拟机 B 和 C 依然通过半虚拟化或设备模拟的方法共享另一个存储设备。通过该方法，虚拟机驱动程序在与设备交互时，不会有任何虚拟机下陷，因而大大减少了因为下陷引起的开销。

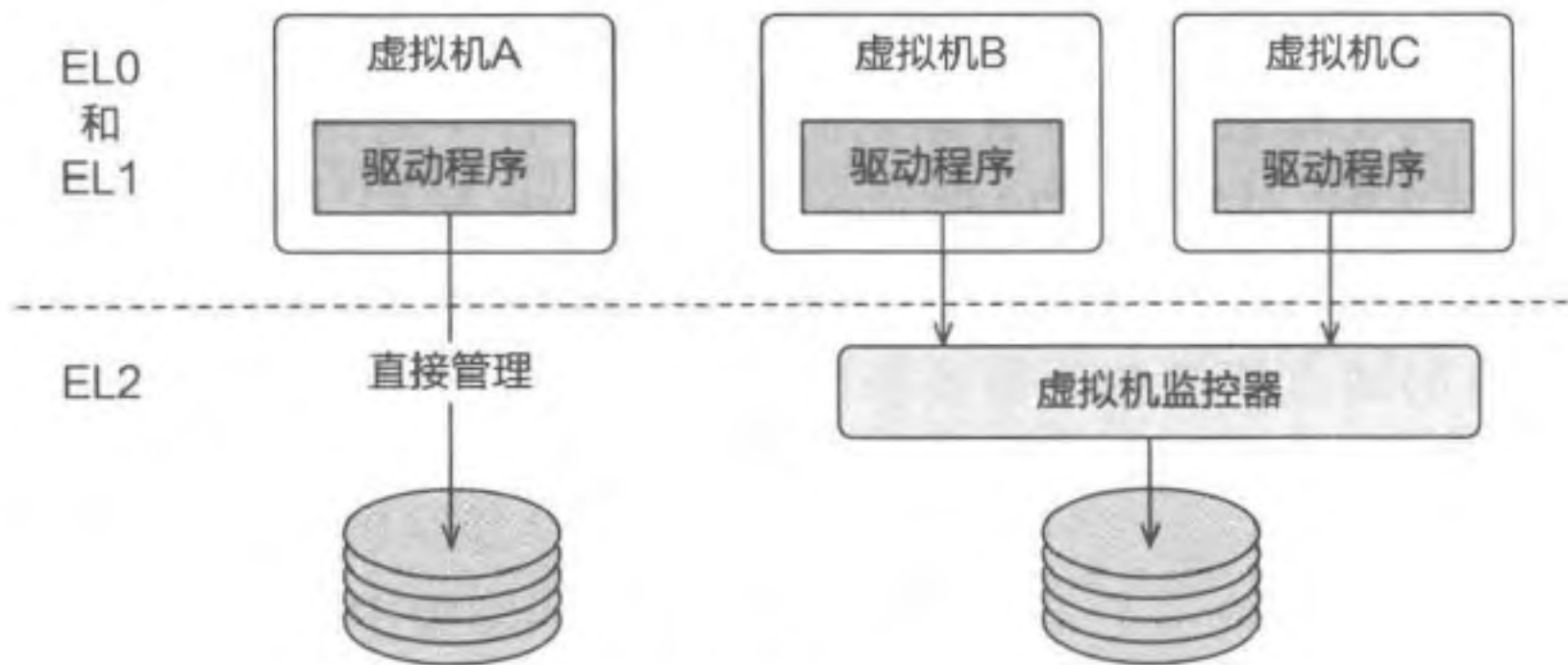


图 14.21 设备直通的 I/O 虚拟化（一）



无论使用全虚拟化还是半虚拟化方法，虚拟机与 I/O 设备交互的数据都需首先拷贝到虚拟机监控器控制的缓存中，之后才会拷贝到设备或虚拟机内。拷贝数据之前，虚拟机监控器需要将客户物理地址翻译成主机物理地址，并检查地址的合法性。然而，在虚拟机和物理设备直接交互时，由于缺少虚拟机监控器的中间翻译，驱动程序在发起 DMA 操作时使用的地址只能是物理设备能够理解的主机物理地址（否则将无法完成 DMA 操作）。

如图 14.22 所示，虚拟机 A 的驱动程序请求一次物理设备的 DMA 操作，要求物理设备将数据拷贝至某个主机物理地址。如果此主机物理地址指向的内容属于其他虚拟机甚至虚拟机监控器，那么此时的 DMA 操作可以恶意读写不属于虚拟机 A 的内存。因此，如果允许虚拟机直接使用主机物理地址与物理设备交互，可能会严重威胁整个系统的内存安全。

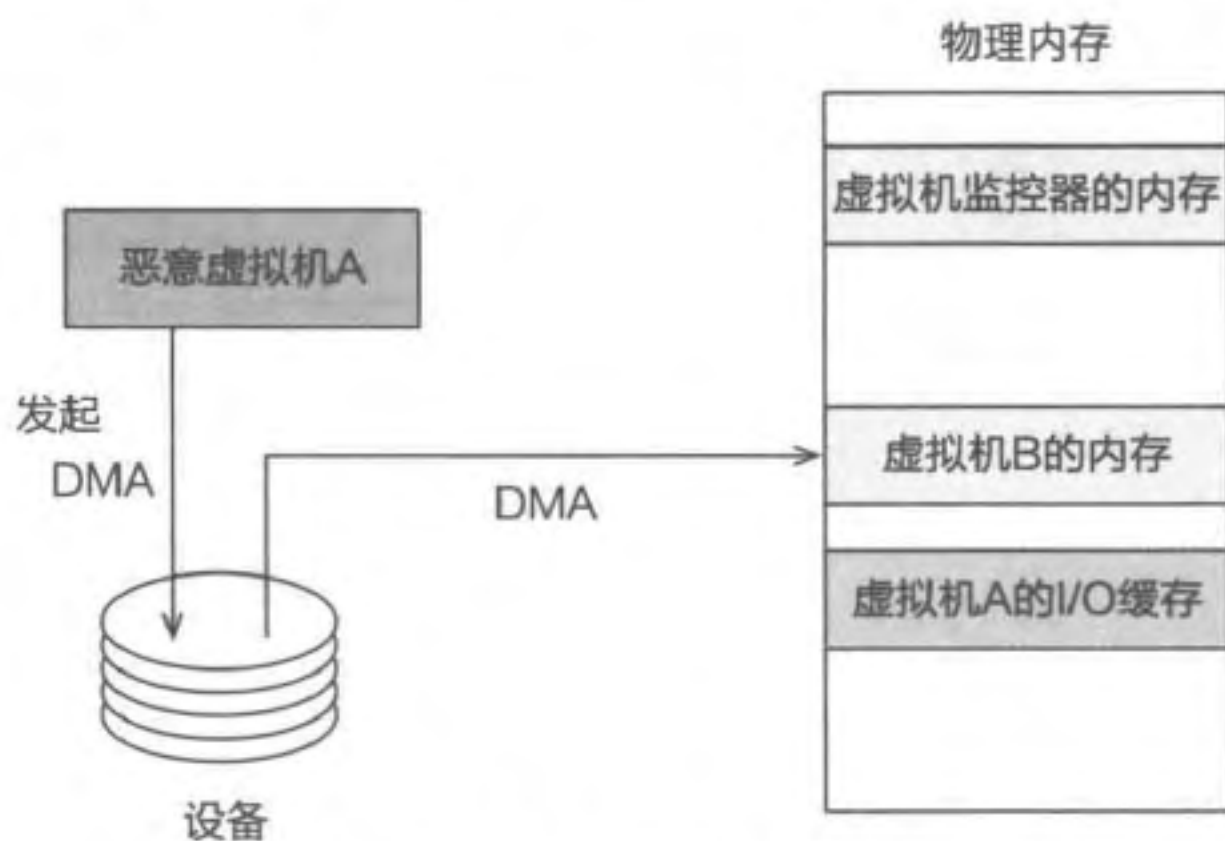


图 14.22 设备直通的 I/O 虚拟化（二）

我们可以使用第 13 章介绍的 IOMMU (I/O Memory Management Unit) 来解决这个问题。与 MMU 类似，IOMMU 也提供第二阶段地址翻译机制，将设备使用的客户物理地址翻译成主机物理地址，并在翻译过程中进行权限检查。为了限制直通物理设备对任意主机物理地址的非法访问，虚拟机监控器首先需要为此设备配置 IOMMU 第二阶段页表，以在页表中维护客户物理地址到主机物理地址的映射关系。在设备进行 DMA 操作时，DMA 的所有客户物理地址都会被 IOMMU 页表翻译成主机物理地址，并进行访存操作。如果恶意的虚拟机试图通过 DMA 访问不属于它的内存区域，IOMMU 翻译时将检查出缺页或权限不足等错误，并通知虚拟机监控器进行处理。

由于 DMA 攻击属于一种通用类型的问题，几乎所有的厂商都提供了相对应的 IOMMU 实现方案以解决该问题，例如 Intel 的 VT-d、AMD 的 AMD-Vi、ARM 的 SMMU 等。我们以 ARM 中的 SMMU (System Memory Management Unit) 为例对 IOMMU 进行解释。与 MMU 存储两阶段页表基地址类似，SMMU 也存储着一个名为 Stream Table 的数据结构，此数据结构包含不同设备的两阶段页表基地址。为了使用 SMMU 的地址翻译机制，虚拟机监控器需要配置 Stream Table 中的页表映射，并将 Stream Table 的基地址记录在 SMMU\_STRTAB\_BASE 寄存器中。Stream Table 为每一个设备都维护了两阶段页表基地址，其以表的结构进行组织，表中每项对应一个设备，名为 Stream Table Entry (STE)，如图 14.23 所示。STE 中的 S2TTB 指向此设备所使用的第二阶段页表基地址。STE 中的 S1ContextPtr 是一组 Context Descriptor 列表的指针，每个 Context Descriptor 包含两个第一阶段页表基地址（与 AArch64 中的 TTBR0\_EL1 和 TTBR1\_EL1 类似）。



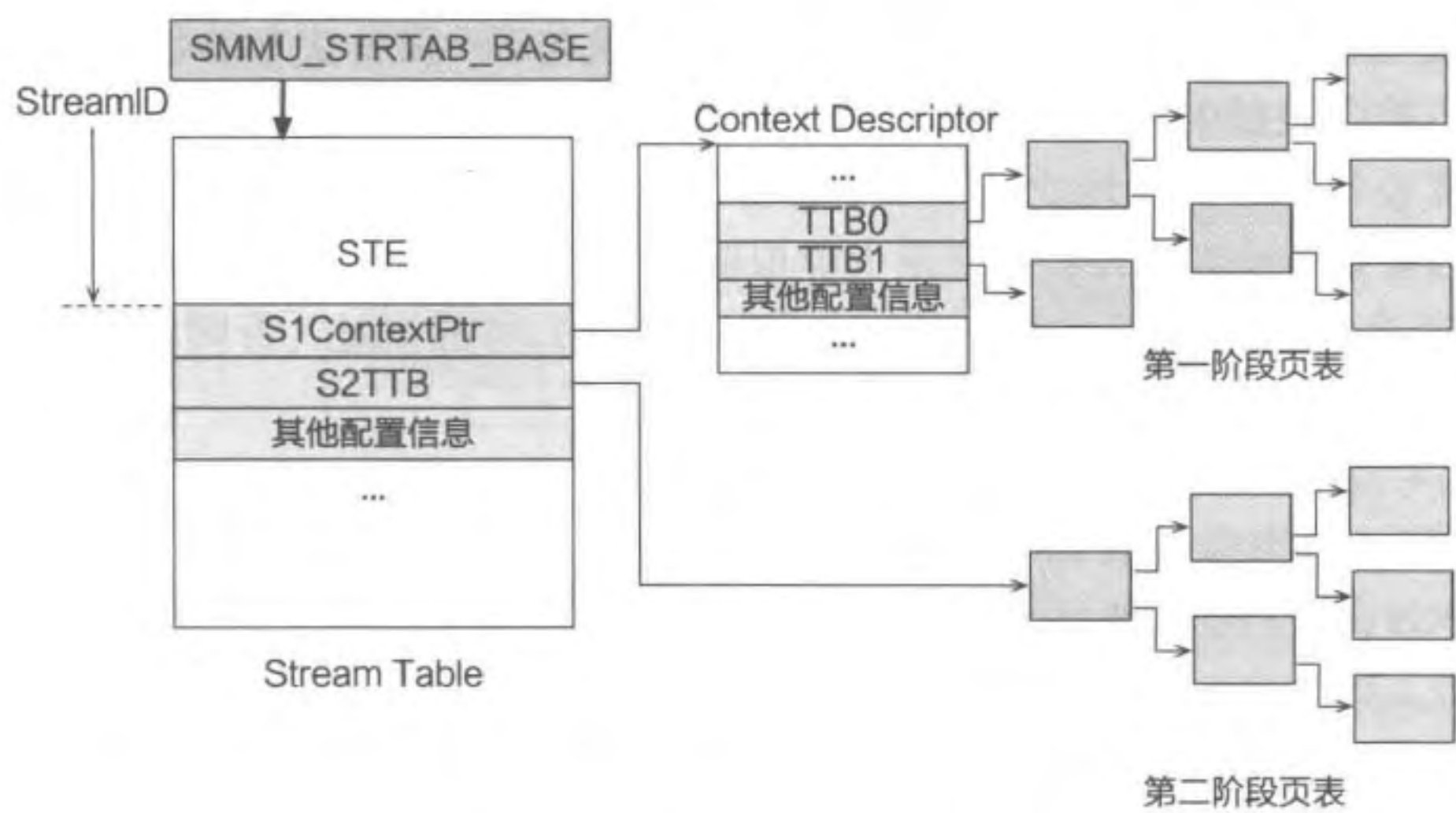


图 14.23 SMMU 中 Stream Table、STE、Context Descriptor 的关系<sup>[5]</sup>。其中使用的  
第一阶段和第二阶段页表结构与前文提到的页表结构完全一致

虚拟机监控器在将物理设备透传给虚拟机直接管理前，需要根据情况配置第二阶段页表，在此页表中限定该设备能访问的主机物理地址范围以及相关权限。图 14.24 展示了 SMMU 进行地址翻译的简要步骤：

- 第一步：驱动发起一次 DMA 操作，并指定此次 DMA 所需访问的内存地址<sup>Ⓐ</sup>、大小、属性（例如读操作还是写操作）。
- 第二步：SMMU 使用设备标识符定位 Stream Table 中对应的 STE。
- 第三步：根据 STE 的信息得到第一和第二阶段页表基地址<sup>Ⓑ</sup>。
- 第四步：进入地址翻译阶段，首先根据 DMA 地址等信息查询 TLB。若 TLB 中已经存在翻译完的映射，就直接得到最终的主机物理地址，直接跳到第六步；若发生 TLB 不命中，则进入第五步。
- 第五步：根据页表和 DMA 地址进入相应的地址翻译过程，并得到主机物理地址。
- 第六步：根据主机物理地址读取或写入内存。

在本节最开始介绍的设备直通场景里，我们假设虚拟机可独占一个完整的物理设备。然而，这种方式在实际生产环境中是很难发生的，因为其有三个问题。首先，如果允许每台虚拟机独占一个物理设备，这会大大提高经济成本。其次，一台物理主机中能够运行的虚拟机数目通常较多，而物理主机中能支持的物理设备的数目少于虚拟机个数，这意味着某些虚拟机不能享受设备直通的优越性能。最后，允许一台虚拟机独占物理设备

Ⓐ 可能是虚拟地址也可能是客户物理地址。  
Ⓑ 在实际使用时，SMMU 可只使用第一阶段翻译或只使用第二阶段翻译，也可以两者同时使用。这是由 STE 中的配置信息决定的。



不利于提高物理资源的利用率，虚拟机一般无法使用完物理设备的所有资源，例如虚拟机很难每时每刻都消耗掉一张网卡的所有带宽。我们该如何解决这些问题？答案就是使用 SR-IOV（Single Root I/O Virtualization）技术。

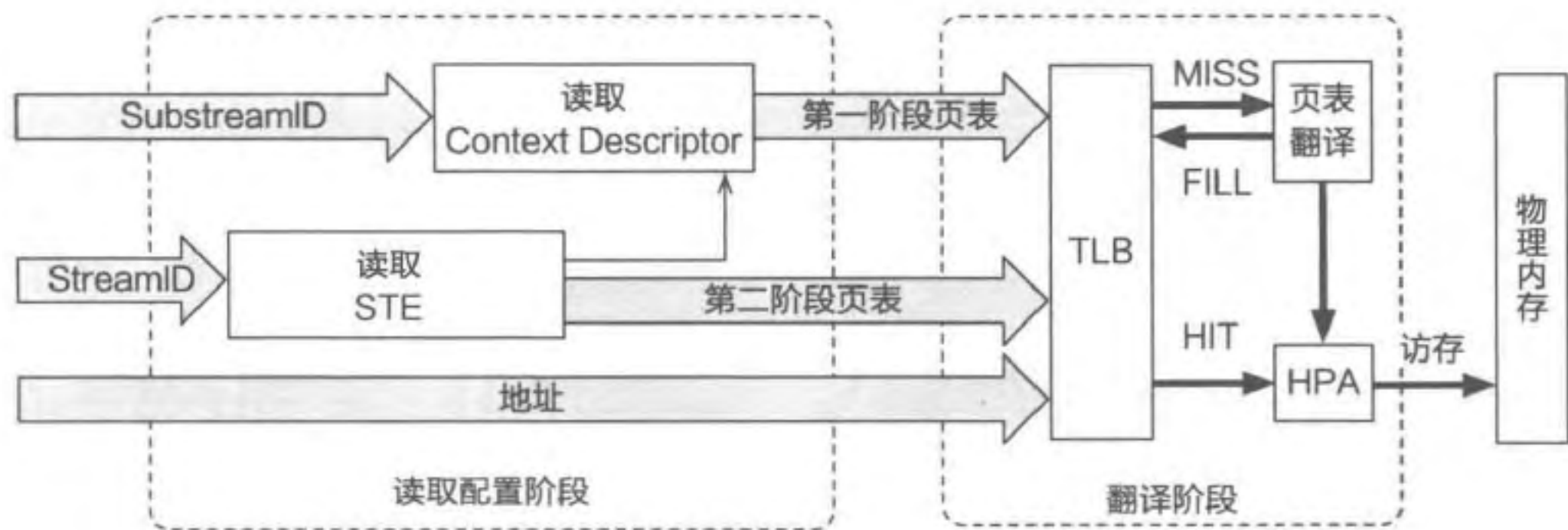


图 14.24 SMMU 中的地址翻译过程<sup>[5]</sup>

SR-IOV 是一个标准化规范，满足 SR-IOV 规范的设备能够在硬件层面实现 I/O 虚拟化。具体来说，该物理设备可创建一个“真身”和多个“分身”，“真身”称为 Physical Function（PF），“分身”称为 Virtual Function（VF）。虚拟机监控器使用 PF 创建和管理 VF，并将每个 VF 直通给一台虚拟机管理。虚拟机通过 DMA、MMIO 等方式与 VF 直接交互，在数据传输的过程中不会下陷到虚拟机监控器中。设备在硬件层面实现 VF 之间的隔离，虚拟机监控器也需要使用 IOMMU 限制每个 VF 能访问的主机物理地址。

在使用 SR-IOV 和 IOMMU 之后，每台虚拟机可独占式地使用直通设备，在数据传输过程中避免虚拟机下陷，从而提升 I/O 虚拟化的性能。然而，由于 I/O 过程完全绕开虚拟机监控器，虚拟机监控器无法在任意时刻检查虚拟机的状态，因而难以支持虚拟机热迁移以及虚拟机安全自省等功能。

14.5.4 小结

表 14.5 总结了本节介绍的三种 I/O 虚拟化方法，基于软件的技术包括软件模拟与半虚拟化，基于硬件虚拟化的技术为设备直通。图 14.25 进一步展示了四种 I/O 虚拟化方法的架构对比（其中半虚拟化方法分成两种子类型）。软件模拟方法（图 14.25a）作用于硬件指令层，因而可以透明地提供对虚拟 I/O 设备的支持。由于大量的虚拟机下陷以及无法批量传输大量 I/O 数据，该方法的性能并不理想。半虚拟化方法通过主动降低透明性来换取性能的提升（图 14.25b），也可以通过将后端驱动直接运行在内核态来进一步提升性能（图 14.25c）。与基于软件的方法不同，设备直通可以安全而高效地将某个物理设备以 VF 的形式直通

表 14.5 I/O 虚拟化技术总结

| 软件技术   | 硬件虚拟化技术              |
|--------|----------------------|
| 软件模拟方法 | 设备直通（IOMMU 和 SR-IOV） |
| 半虚拟化方法 |                      |

给一台虚拟机使用，从而进一步提升 I/O 虚拟化的性能（图 14.25d）。

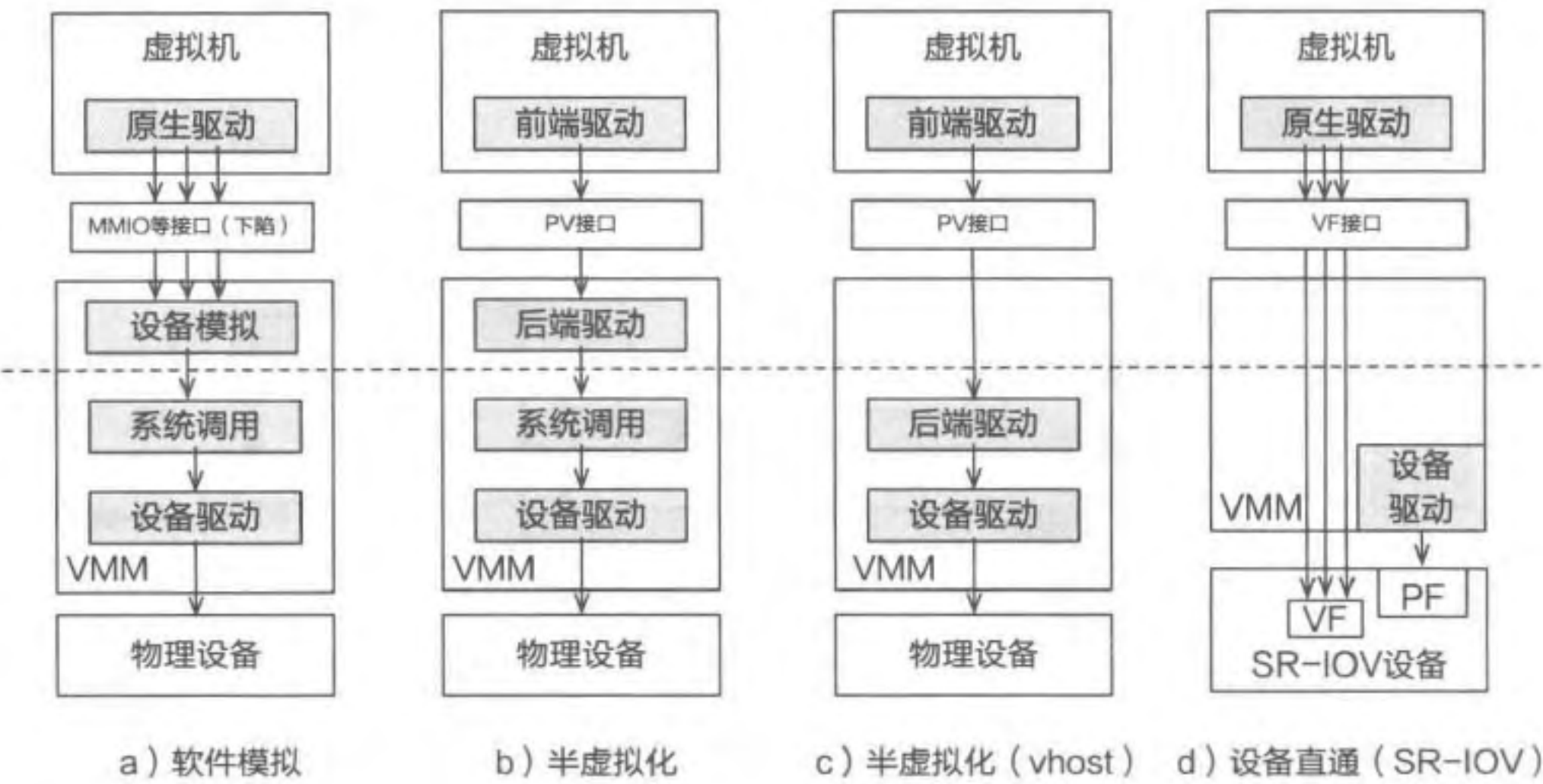


图 14.25 I/O 虚拟化方法的架构对比

## 14.6 中断虚拟化

### 本节主要知识点

#### □ 如何在 AArch64 架构中实现中断虚拟化？

中断是操作系统与设备交互时必不可少的机制。在完成某任务后，设备可通过中断异步地通知 CPU，以便其进行后续操作。例如，当网卡设备接收到网络包时，它可发送中断异步地通知 CPU，CPU 随后调用操作系统注册在 VBAR\_ELx 寄存器中的中断处理函数。

在非虚拟化场景中，来自设备的中断直接唤醒操作系统配置的中断处理函数。虚拟化场景中的中断传输情况稍显复杂，分为两种类型，一种是物理中断，一种是虚拟中断。物理中断由物理设备直接产生，交给虚拟机监控器处理。虚拟机监控器一般不允许设备直接将物理中断发送给虚拟机，这主要是因为物理设备的中断不一定和某个虚拟机直接相关，例如，有可能此时网卡收到的网络包属于另一个没有运行的虚拟机。虚拟中断不由物理设备直接产生，而由虚拟机监控器（或下文介绍的 ITS）产生并传递给虚拟机。

在推出硬件虚拟化技术之前，中断虚拟化需通过软件模拟的方式实现。一方面，软件模拟方法中的所有中断寄存器本质上都是内存中的状态，这些虚拟寄存器的状态改变需要由虚拟机监控器维护。对于虚拟机而言，寄存器所在的内存必须被设置为不可访问。